Q-Relay: presentation script

Team: Maruf · Parag · Vaibhav **Event:** Qiskit Fall Fest 2026, DBATU Lonere **Length:** 8 minutes + Q&A

Who	Owns	Time
Maruf	Opening, Mission Control (the whole story), how it all connects, closing	0:00–2:15 and 7:15–8:00
Parag	Problem 2: random numbers (QRNG Lab) and Problem 1: teleportation (Teleport Lab)	2:15–5:00
Vaibhav	Problem 3: Deutsch–Jozsa (DJ Lab), the judge game, hardware and honesty	5:00–7:15

How to read this script. Coloured boxes are what you **say**. Brown lines in [brackets] are what you **do** on the laptop. Whoever is speaking holds the mouse. At each handoff, slide the laptop or mouse to the next person. You don't have to memorise word for word: the **bold** phrases are the key points not to miss.

Before the judges arrive (2 minutes)

- Start the app: `.\tasks.ps1 demo`, then open **http://127.0.0.1:8000** in Chrome or Edge and press **F11** (full screen).
- Turn the laptop volume to about 30%. Music starts after the first click.
- Wait on the **title screen**. If the laptop is slow, press **L** (low-power mode). On a projector, press **P** for big captions.
- Have **notebooks/** open in a second window (VS Code or Jupyter), in case a judge asks to see the code.
- Emergency: if anything freezes, press **Esc** (title menu) and then **D** (90-second automatic demo). It narrates everything by itself.

Part 1: Opening (Maruf · 0:00–0:45)

[Title screen is showing, music playing softly.]

> **Maruf:** Good morning. We are Maruf, Parag and Vaibhav, and this is **Q-Relay**, a tiny quantum internet running in a browser.
>
> The fest gave us three problems: **build a quantum random number generator, teleport a qubit, and run the Deutsch–Jozsa algorithm.** Most teams solve them as three separate notebooks. We did something different: we **chained all three into one mission**, where each stage needs the one before it.
>
> Everything is built with Qiskit, everything is tested (**53 automated tests**), and every number on screen is **labelled with where it came from**: simulator, emulated IBM chip, or real hardware.

[Click **NEW MISSION**. If the Mission Guide pops up, click **Skip**.]

Part 2: Mission Control (Maruf · 0:45–2:15)

[On Mission Control. Set **n = 3**, speed **1×**, noise **0%**. Click **LAUNCH MISSION**.]

> **Maruf:** The mission has three stages. Watch the progress rail at the top, and read the narration at the bottom.
>
> **Stage 1:** a quantum random number generator fills an entropy pool. Those random bits secretly choose **a state for Alice to send** and **a hidden puzzle for Bob to solve.**

[When the Bell pair forms, point at the small purple ball and Bob's ball.]

> **Maruf:** **Stage 2:** Alice and Bob share an **entangled pair**. Look at the arrows: they **shrink to nothing**. That's not a bug. Half of an entangled pair, on its own, is completely uncertain, and we draw that honestly.

[When the two little cubes fly along the wire.]

> **Maruf:** Alice measures and sends Bob **two ordinary bits**, those little cubes. Bob uses them to fix his qubit, and his arrow **lands exactly on the white ghost**, which is Alice's original. Fidelity: **1.000**, a perfect transfer. And Alice's copy is destroyed: **the state was moved, not copied.**

[When the DJ bars appear.]

> **Maruf:** **Stage 3:** Alice teleports one more qubit, and Bob uses it as the helper for Deutsch–Jozsa. **One question to the hidden box**, and we get the right answer.

[After "Mission complete", slowly drag the **Noise** slider from 0% to about 80%.]

> **Maruf:** Real quantum computers are noisy, so here is our noise knob. Watch the **glowing link flicker**, Bob's **arrow shrink**, and even the **music get muddy**. When the fidelity gauge drops **below 2/3**, it turns red: "CLASSICAL REGIME". Two-thirds is the best you can do **without** quantum physics, so below that line the quantum advantage is gone.

[Drag noise back to 0%. Hand the mouse to Parag.]

> **Maruf:** Parag will now show you each stage up close, starting with the randomness.

Part 3: Random numbers, Problem 2 (Parag · 2:15–3:30)

[Press **2** (QRNG Lab). It opens in **Story** mode with the **IBM HW** bit source. Chapter 1 (the coin) is showing. Press → (or Next) through chapters 2–5 as you talk: the counters tick up, the bars grow, the random walk draws itself.]

> **Parag:** The circuit is the simplest in quantum computing: **put a qubit in superposition with a Hadamard gate, then measure it.** It comes out 0 or 1 like a perfect coin. Here we generated **one million bits**.
>
> The bar chart shows roughly 50/50 zeros and ones, inside the orange band that pure chance allows. The random walk stays inside its envelope.
>
> Then we run **ten statistical tests**, taken from the US NIST standard and written ourselves. Green means no pattern found.

[Chapters 6–8: the fingerprints scan in, the ten detectives pop up one by one, then RANDU's stripes appear with its red tests.]

> **Parag:** To prove the tests actually catch bad randomness, we fed them a famously broken classical generator called **RANDU**. It **fails six of the ten tests**, and when you draw its bits as a picture you can **see stripes**. Good randomness looks like TV static.
>
> One honesty point: on a simulator, the "quantum" randomness is really produced by a classical algorithm. **Only bits from real hardware are truly random**, and the app labels that clearly.

[Chapter 9: the 100 hardware qubits as bars, then the before/after fixing chart.]

> **Parag:** Real devices are slightly biased, so we also implemented three **bias-fixing methods**: Von Neumann, XOR and Toeplitz hashing. Starting from a 40/60 unfair source, Von Neumann and Toeplitz bring it back to 50/50 and **pass all 10 tests**. Toeplitz keeps about three times more bits.

Part 4: Teleportation, Problem 1 (Parag · 3:30–5:00)

[Press **3** (Teleport Lab, Story mode). Chapter 1: drag the arrow on the left sphere around.]

> **Parag:** You can **pick any state by dragging Alice's sphere**, and Bob's sphere follows. That's what "arbitrary state" means.

[Press → for chapters 2, 3 and 4: the spheres step through the protocol by themselves, with the glowing gate moving along the circuit.]

> **Parag:** The step slider replays the circuit gate by gate: the Bell pair, Alice's measurement, then Bob's corrections. It's a **dynamic circuit**: the measurement happens in the middle and the result controls the next gates, exactly like the real protocol.

[Chapter 5: the four outcome rows appear one by one; then chapter 6, the 200-state histograms.]

> **Parag:** How do we know it works? Alice's measurement has **four possible outcomes**, and in **every one of the four** Bob's fidelity is **exactly 1**. We check it four independent ways: exact density matrices, our own tomography, an average over **200 random states**, and a full process matrix.

[Chapter 7: the noise curves draw themselves and the circles mark the 2/3 crossings. Then jump to chapter 9 (real IBM results) or leave that for Vaibhav.]

> **Parag:** For noise, we **derived a formula** for the simplest case, and the dashed line from the formula sits exactly on the simulation. The curves cross the **2/3 line at noise of about 0.38 to 0.42**. That's our **quantum-advantage threshold**.
>
> We also found something subtle: when Alice's measured bits are misread, Bob applies the wrong correction, and **no error correction afterwards can undo it**. We measured exactly that ceiling.

[Hand the mouse to Vaibhav.]

Parag: Vaibhav will now show the one-question puzzle.

Part 5: Deutsch–Jozsa, Problem 3 (Vaibhav · 5:00–6:30)

[Press **4** (DJ Lab, Story mode). Chapter 1 shows the box and its secret answers. Press → through chapters 3, 4 and 5: the bars spread out, flip, then cancel, and the verdict pops in.]

> **Vaibhav:** Imagine a sealed box. You give it an input, it answers 0 or 1. We're promised it's either **constant**, always the same answer, or **balanced**, exactly half zeros and half ones. Which one is it?
>
> Watch the bars, one per possible input. **First, superposition:** we ask about all inputs at once, so every bar is equal. **Second, phase kickback:** after one question, the inputs where the answer is 1 **flip upside down**. **Third, interference:** the waves combine. For a balanced box, the bar at 000 **cancels to exactly zero**, and for a constant box all the height piles onto it.

[On the left, change **box** to **const_1** to show the opposite case; then go back to chapter 2 (the race and the growth chart).]

> **Vaibhav:** A normal computer might need **half the inputs plus one**: for 20 bits, more than half a million questions. Quantum: **one**. We even simulated an adversary box that tries to fool a classical algorithm, and it needs exactly that many. And we checked **every possible function** for up to 3 bits: all 80 classified correctly.

[Press **5** (Beat the Oracle). Offer the mouse to a judge.]

> **Vaibhav:** Would you like to try? Click tiles to ask the box, the normal way. *(Let them click 3 or 4 tiles.)* To be **certain** you might need up to 17 clicks. Now let's ask the quantum computer.

[Click **Ask the quantum computer**.]

> **Vaibhav:** One question, and the answer is **teleported to Bob's screen** with its fidelity shown.

Part 6: Hardware and honesty (Vaibhav · 6:30–7:15)

[Press **6** (Hardware).]

> **Vaibhav:** Here, simulator, **emulated IBM device** and **real hardware** sit side by side, each with a badge. And yes, we ran it on a **real IBM quantum computer**, ibm_marrakesh, job ID shown right here.
>
> Teleportation on real hardware: average fidelity **0.978**, far above the 2/3 classical limit. With true mid-circuit measurement it's **0.880**, still clearly quantum; the drop happens because the qubit loses phase while it waits for the result. Deutsch–Jozsa got **all 10 circuits right**, and we collected **819,200 physically random bits**. The detectives even caught a tiny real bias in them, which our extractors then fixed.
>
> Everything is backed by **four executed Jupyter notebooks**, a written report with derivations, and **53 automated tests**.

Part 7: The connection and closing (Maruf · 7:15–8:00)

[Press **1** (Mission Control).]

> **Maruf:** Our favourite result ties the problems together. In Stage 3, Bob's helper qubit is **teleported**. We proved, and confirmed in simulation, that the chance of correctly detecting a balanced box is **exactly equal to how well that helper was teleported**. So **teleportation quality directly limits the algorithm's correctness**: one number links Problem 1 to Problem 3, and Problem 2 supplies the randomness for both.
>
> So that's Q-Relay: **all three problems solved, connected, tested and honestly labelled**, and it runs completely offline. Thank you. We're happy to take questions.

[Leave Mission Control on screen. Optional: press **D** during Q&A so the demo loops in the background.]

Q&A: who answers what

Question topic	Answers
Why 2/3? Is the simulator truly random? Why can't you clone? Faster than light?	**Parag**
How does Deutsch–Jozsa work? Query counts? What does noise do to DJ?	**Vaibhav**
Architecture, the code, the web app, the "teleported helper" result, real hardware	**Maruf**

Crisp answers to practise:

- **Why 2/3?** Without entanglement, the best you can do is measure the unknown state once and describe it over the phone. The best average score for that is 2/3. Above 2/3 proves the channel is truly quantum.
- **Is simulated randomness real?** No. The simulator uses a classical random generator, and we say so on screen. Only hardware bits are physically random. Statistical tests find patterns; they cannot prove where randomness came from.
- **Why can't Alice just copy the qubit?** The no-cloning theorem. In our step view you can see Alice's qubit collapse the moment she measures.
- **Is teleportation faster than light?** No. Bob needs Alice's two ordinary bits, and until they arrive his qubit is pure noise.
- **Why a dynamic circuit?** Bob's correction depends on a measurement made in the middle of the circuit. That is the real protocol. We also built a version without mid-circuit measurement for hardware that doesn't support it.
- **What does noise do to Deutsch–Jozsa?** The answers drift toward random. Deeper boxes fail first. At very high noise "balanced" looks right by pure luck, so we also report a fairer measure.
- **How do you know your numbers are right?** Four independent methods agree, a hand-derived formula matches the simulation to six decimal places, and there are 53 automated tests.

If you only get 3 minutes

1. **Maruf (60 s):** opening line and one Mission launch at speed **4×**. Mention the shrinking arrows, Bob landing on the ghost, and fidelity 1.
2. **Parag (60 s):** QRNG Lab, **Break the PRNG** (stripes), then the teleport noise chart and the 2/3 line.
3. **Vaibhav (45 s):** DJ Lab Replay (three phases), then "1 question vs 17".
4. **Maruf (15 s):** "One noise knob affects all three stages; teleportation quality limits the algorithm. Thank you."

Backup plans

Problem	Fix
Red "Backend offline" box	In a terminal: `.\tasks.ps1 demo`, then refresh the page
Laptop lagging	Press **L** (low power)

Problem	Fix
A lab looks crowded or a judge wants every chart at once	Click **OVERVIEW (all panels)** at the top right of the lab
Want the lab to narrate itself	Click ▶ **Play story** (chapters advance automatically)
Mission seems stuck	Press **Esc** then **NEW MISSION**, or press **D** for the automatic demo
No sound	Click anywhere once (browsers need a click), or press **M**
Projector text too small	Press **P** for presenter captions, or Ctrl + and Ctrl − to zoom